SynthGraph

Workspace performance: before and after

Observed on production, fixed in two deploys (379c83c: tab clicks and rate limiting; 1b1eb67: one workspace request), and re-measured on production after each with the same script. Local comparisons cover the failure cases that should not be provoked on the live site.

Bottom line

Every tab click on production re-ran the whole server page: about 33 backend calls and a 260-400 KB payload, and the screen only changed after 1.4-2.6 s (median 1.9 s; 8 clicks measured today).

On production after the deploy, the same 8 clicks take 8-24 ms (median 13 ms) and make zero server requests; a burst of 30 clicks leaves the Reproduction tab with 16 of 16 manifests loaded. That is about 150 times faster.

After the first deploy a full page load was still about 2.0 s, because the three sequential backend hops were unchanged. The second deploy loads the whole workspace in one backend request: the server render fell from 2015 ms to a median of 871 ms, and Refresh data from 2.1 s to 1.3 s. A hard page load is about 1.6 s (median of three, range 1.1-2.0 s).

Two reliability faults the slowness was hiding are also fixed: one user's traffic could rate-limit everyone else, and a single rate-limited request could sign a valid user out.

1. What I did

- Production, before (read-only): signed-in test account, Phase 1 experiment (16 generations, 3 runs). Clicked the six workspace tabs repeatedly with a timing script, took screenshots, read Resource Timing for every request.
- Implemented the fixes on a new branch from origin/main and committed them (379c83c). You merged and deployed it. After the first production re-check the full page load was still about 2 s, so I added a second change (1b1eb67: batched health and drift, one workspace endpoint, timing), which you also merged and deployed.
- Before the deploy, re-measured on local production builds against a mock backend shaped like the production experiment. The old build (origin/main) and the new build ran against the same mock, so that comparison is like for like. After the deploy, repeated the exact production script from section 2 (section 5).

Read this before trusting the numbers

Production "before" and "after" numbers are real (synthgraph.dev, 2026-10-30, same account, same experiment, same eight clicks). Local numbers (section 4) come from a mock backend with 60 ms per call and no network, so their absolute milliseconds are NOT comparable to production; they are there for the request counts and the failure cases.

Not exercised on production: the rate-limit failure path (nothing got throttled) and the per-user throttle (provoking it would stress the live site). Both were verified locally only.

The second change was also measured on a real backend and database with a proxy adding a 150 ms round trip (section 6). That shows the effect of round trips and call counts, not production milliseconds.

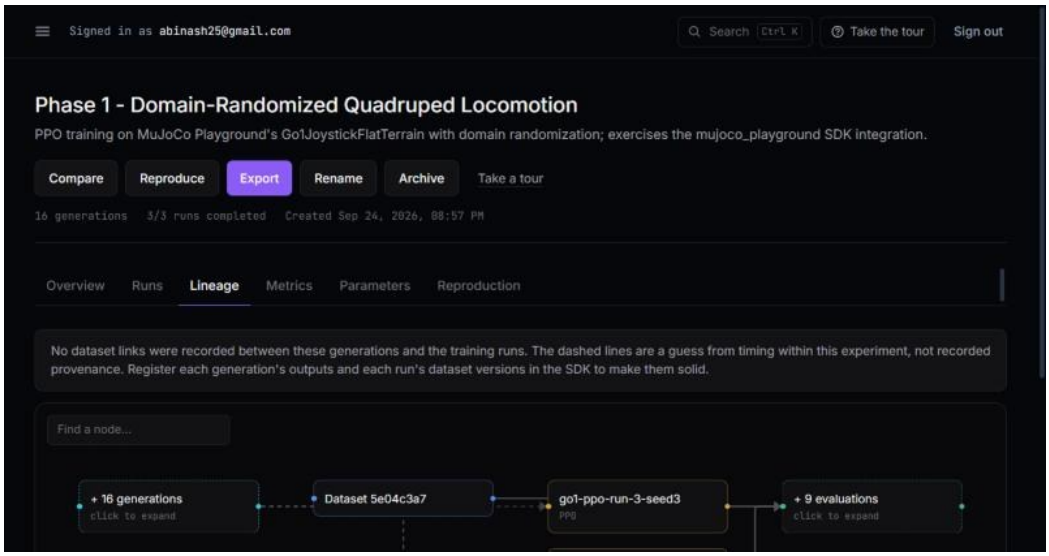
2. Before: what production showed

Method: a script clicks a tab, waits until the highlighted tab and the ?tab= URL both match, and records that time, then reads Resource Timing for the requests the click caused. Same script, same experiment, run today.

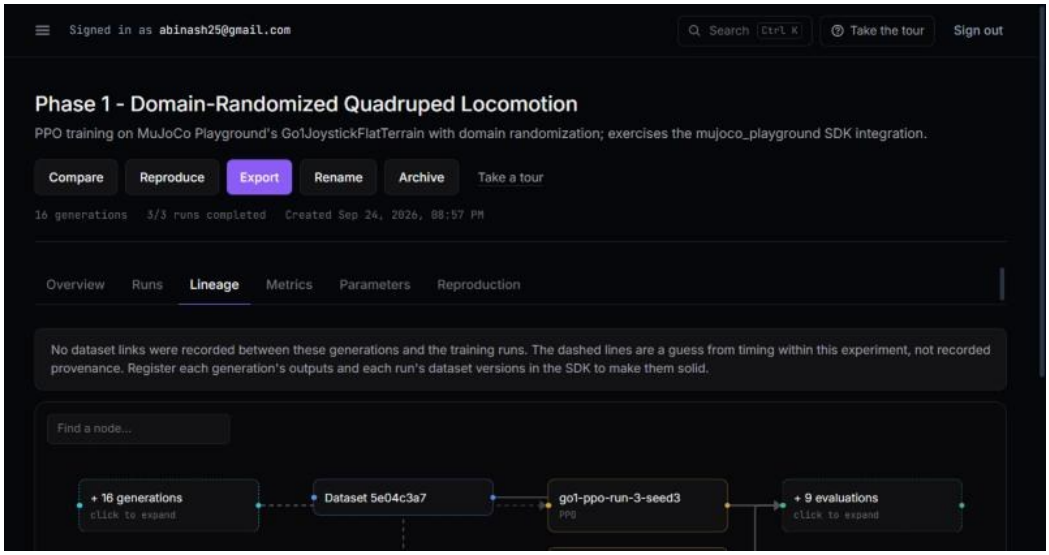
Click	UI changed after	Server requests	Payload (decoded)	Note
Runs	2229 ms	1 (RSC)	390 KB	time to first byte 227 ms; the rest is waiting on the server render
Lineage	2111 ms	1	390 KB	first byte 535 ms

Click	UI changed after	Server requests	Payload (decoded)	Note
Metrics	1869 ms	1	390 KB	
Lineage (again)	1917 ms	1	390 KB	revisiting a tab is not faster
Parameters	2601 ms	1	390 KB	slowest of the eight
Reproduction	1926 ms	1	390 KB	
Overview	1587 ms	1	327 KB	
Lineage (again)	1387 ms	1	256 KB	fastest; payload shrank because the rate limit started dropping manifests

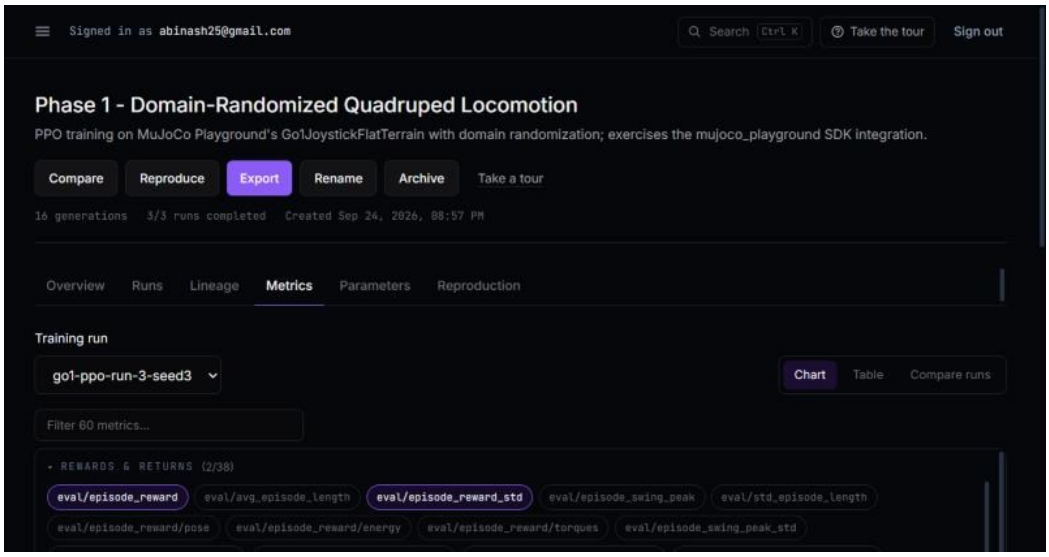
Median 1922 ms, range 1387-2601 ms. The compressed transfer is only about 25 KB per click and first-byte time is about 190 ms, so the cost is not the network. It is the server re-running the whole page for a click that only needs data the browser already has.



Production, Lineage tab loaded. Next step: click Metrics.



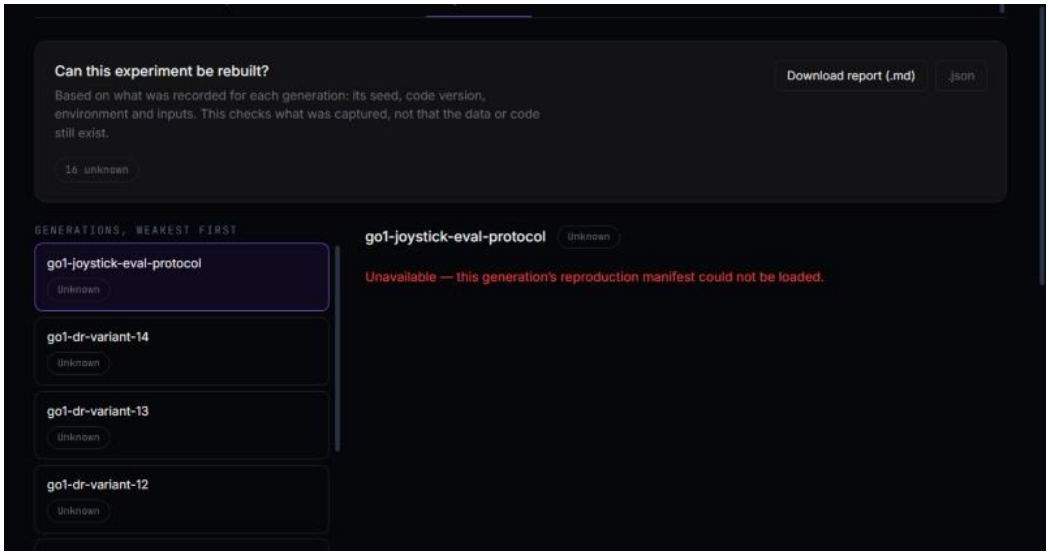
Production, captured right after clicking Metrics. The URL is still ?tab=lineage and the Lineage canvas is still on screen: the click has not done anything visible yet.



Production, about 3 seconds later: the Metrics tab has finally appeared (URL now ?tab=metrics).

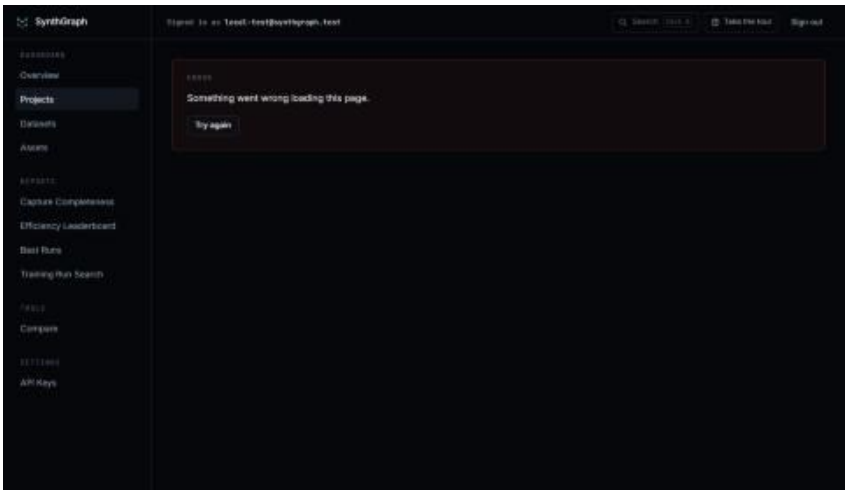
The silent failure behind the slowness

Right after the click burst, the Reproduction tab showed every generation as "Unknown" with "this generation's reproduction manifest could not be loaded." The backend had rate-limited the manifest requests (120 per minute per IP, and one page render uses about 33), and the page swallowed the error. It looked like a property of the data, not a loading failure.

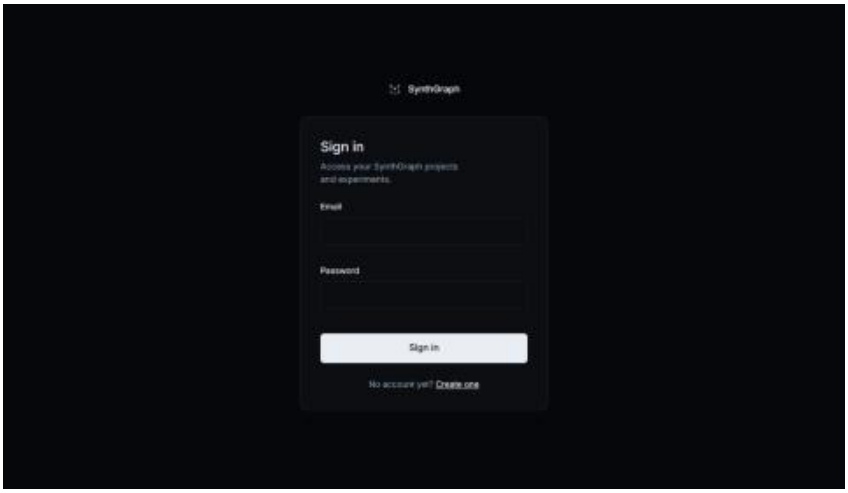


Production: 16 of 16 generations "Unknown" because the manifests were rate-limited, not because anything is wrong with the generations.

The same limit reproduced on the old build against the mock. After about four renders in a minute, the next one was refused, including the sign-in check, and the app treated that as "logged out":



Old build, local mock: a rate-limited core call turns the whole workspace into "Something went wrong".



Old build, local mock: a different user sent 200 requests; this user, who did nothing, was bounced to the sign-in page. The old limit counts per IP, and every dashboard user reaches the backend from the same frontend server.

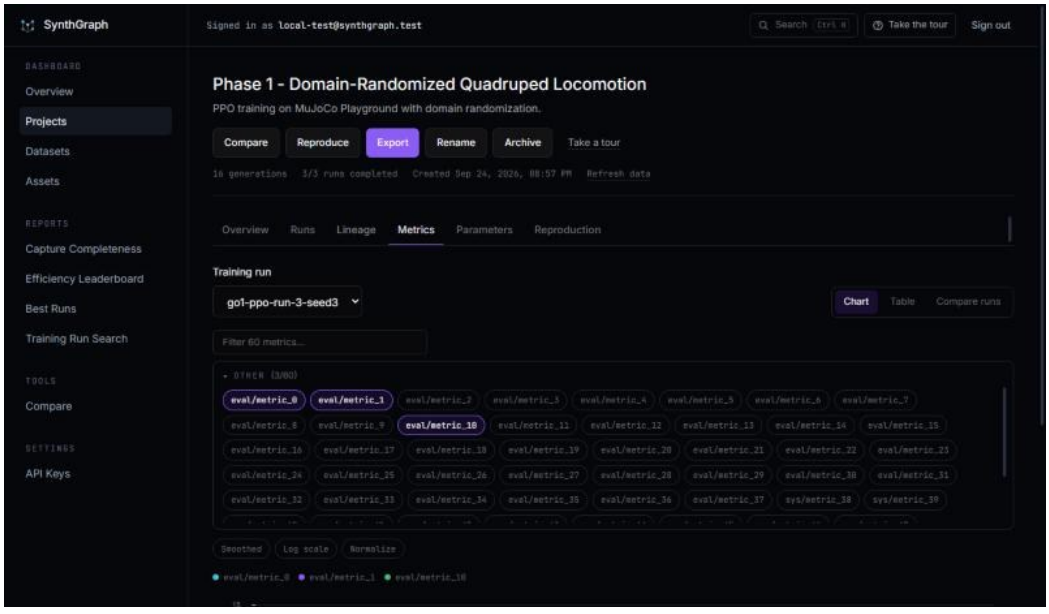
3. What I changed

Issue	Problem	Change
PERF-01	Tab and selection clicks re-ran the server page	Switch the URL with history.pushState (Next syncs useSearchParams), so no server request is made. Back/forward still work. A "Refresh data" link reloads on demand. shared/lib/shallow-url.ts, Tabs.tsx, ExperimentWorkspace.tsx, ExperimentHeader.tsx.
PERF-02	33 backend calls per render	New additive backend route GET /experiments/:id/reproduction-manifests returns all manifests in one request (one query for dataset references). The frontend uses it and falls back to per-generation fetches if the backend is older. 33 calls down to 18 per render.
PERF-03	One user's traffic rate-limited everyone; one 429 signed users out	Rate limit now counts per bearer credential (hashed), with a 3000/min per-IP ceiling; login and register stay per-IP. getSession only treats a rejected key (401/403) as logged out; any other failure shows an error instead of deleting the cookie. Reads retry briefly on a short 429.
PERF-04	Failed manifests shown as "Unknown"	Each generation records manifestError (rate_limited or failed). The Reproduction tab shows a banner with a Retry button and per-row "Rate limited" chips, and counts them apart from "unknown".
PERF-05	Duplicate payload	The unused normalized mirror (generation and manifest) is no longer sent to the browser. Source text duplicated in parameters remains; see limits.

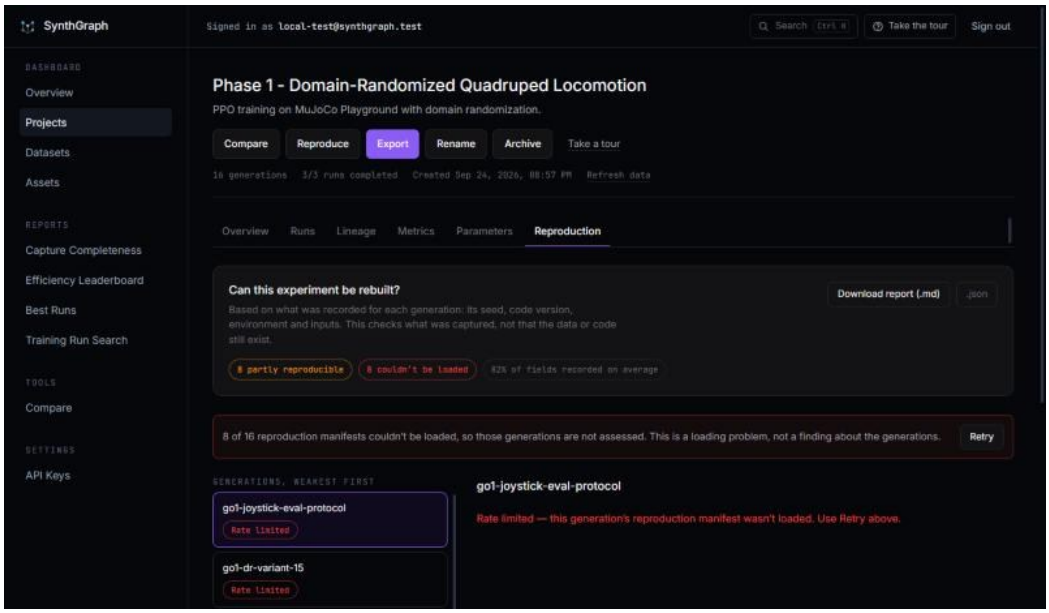
Issue	Problem	Change
PERF-07	No server-side timing	The page logs one experiment_workspace.loaded line with duration, counts and manifest failures.
PERF-08	Animation added a fixed delay	Tab and inspector transitions: no exit phase, 120 ms fade-in (was exit 200 ms + enter 200 ms).

4. After: local production build, same mock (failure cases)

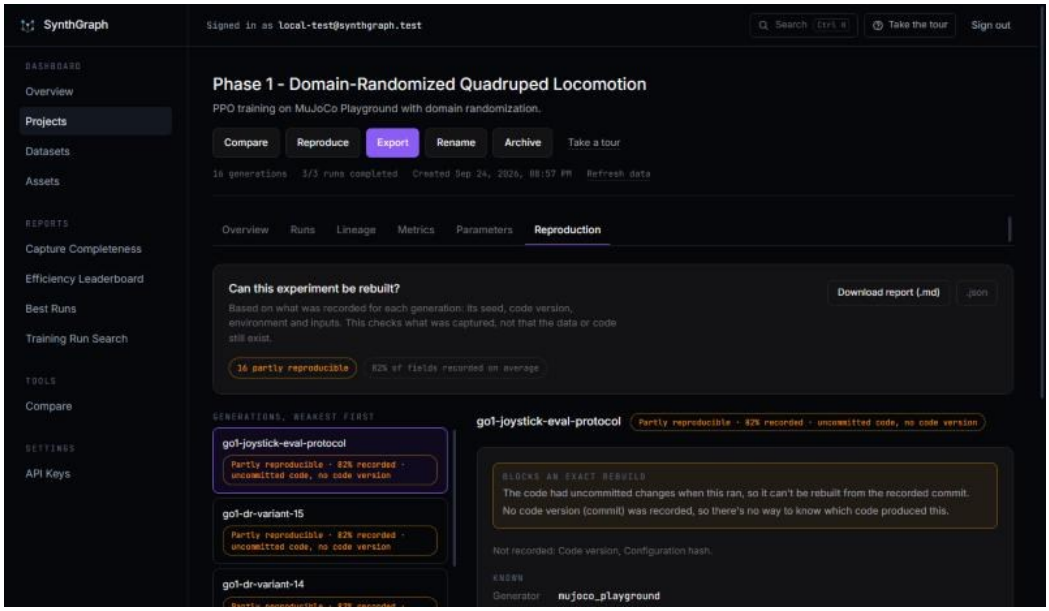
Measure	Before (old build)	After (new build)	Change
Tab click to UI changed	261-295 ms (3 clicks, then failed)	6-31 ms, median 14 ms (24 clicks)	**about 20x faster, and no server wait**
Server requests per click	1 render request	0	**-100%**
Backend calls per click	33-34	0	**-100%**
Backend calls per workspace render	33	18	**-45%**
Page payload (decoded)	521 KB	435 KB	**-17%**
normalized mirrors in payload	32 (16 + 16)	0	**removed**
Another user sends 200 requests	this user signed out	no effect, 16/16 manifests loaded	**fixed**
Rate-limited manifests	shown as Unknown	banner + Retry; Retry loads all 16	**fixed**
Backend restarts mid-session	sign-in check failure = logged out	error shown, session kept	**fixed**



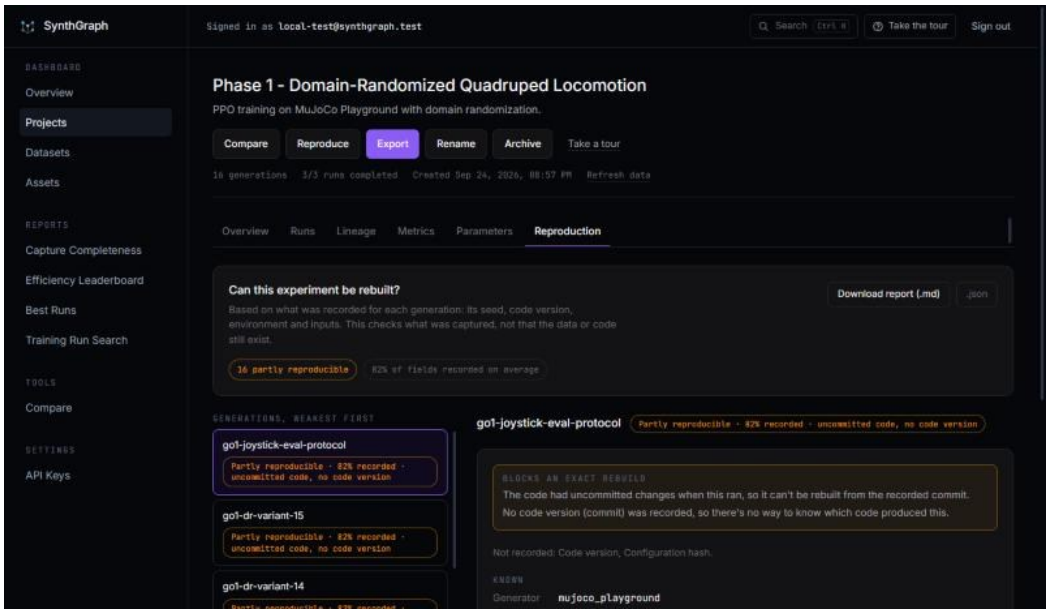
New build: Metrics tab immediately after the click; the URL and content change together, with no request.



New build, with 8 of 16 manifests forced to rate-limit: an explicit banner, a "couldn't be loaded" chip, per-row "Rate limited", and a Retry button.



After clicking Retry: all 16 generations assessed.



New build, right after another user sent 200 requests: this user's Reproduction tab loads normally.

5. After the first deploy: production, same script as section 2

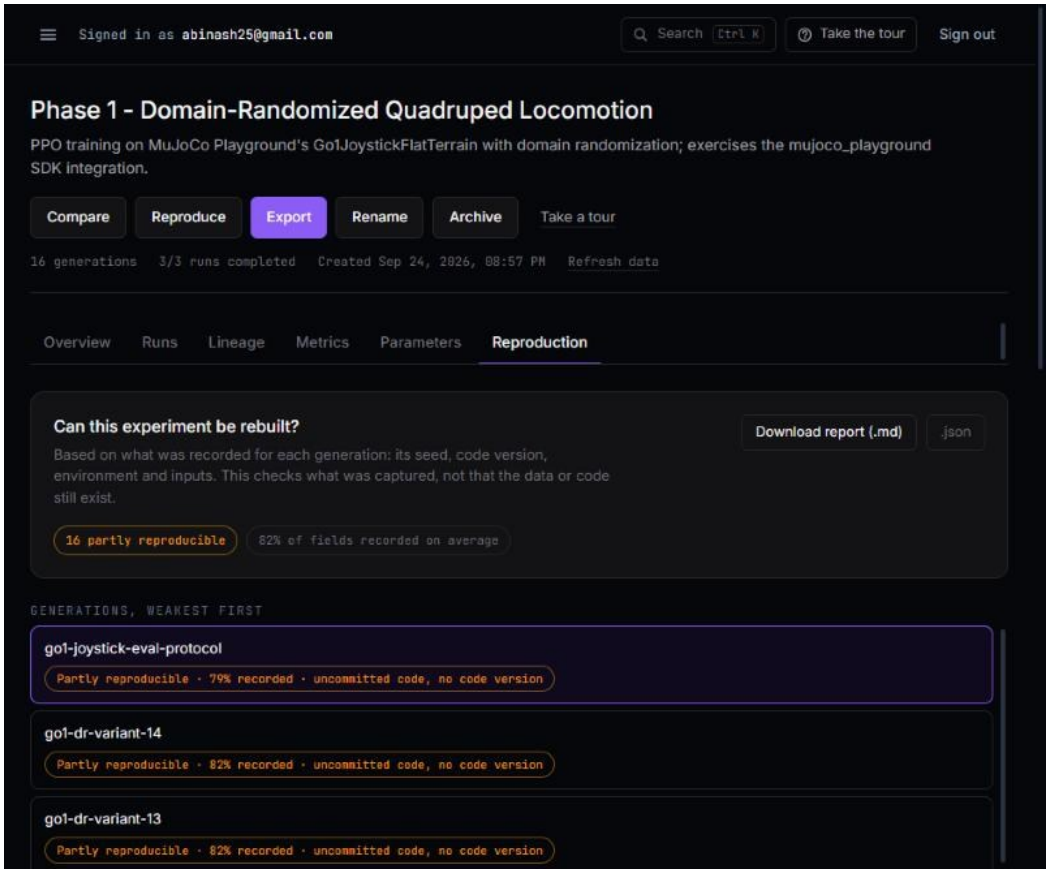
Same account, same Phase 1 experiment, same eight clicks, run after the deploy of 379c83c.

Click	UI changed after	Server requests	Before	Note
Runs	24 ms	0	2229 ms	
Lineage	8 ms	0	2111 ms	
Metrics	22 ms	0	1869 ms	
Lineage (again)	14 ms	0	1917 ms	a revisit is as fast as a first visit
Parameters	23 ms	0	2601 ms	slowest before; now 23 ms
Reproduction	11 ms	0	1926 ms	
Overview	11 ms	0	1587 ms	
Lineage (again)	12 ms	0	1387 ms	

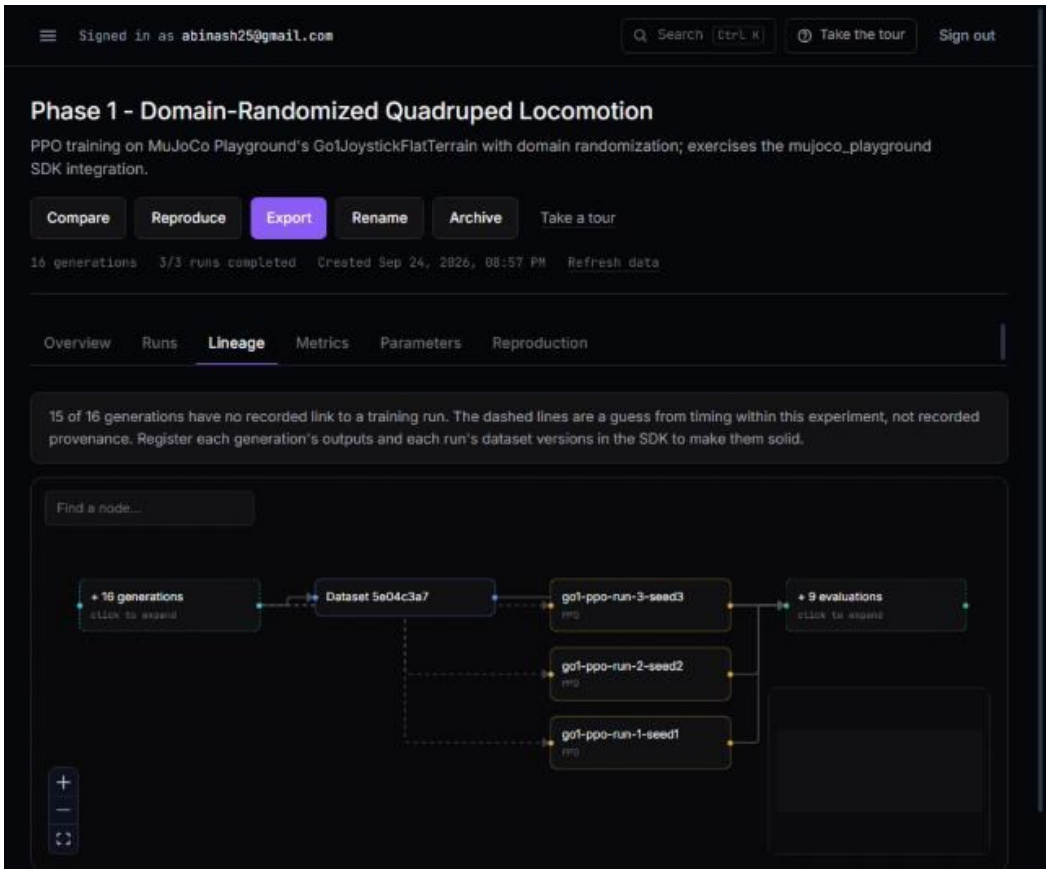
Median 13 ms, range 8-24 ms (before: median 1922 ms, range 1387-2601 ms). The only requests in the page after the clicks are the sidebar link prefetches from the initial load; the clicks themselves added none.

Measure	Before (production)	After (production)
30 clicks, 40 ms apart	not run; 8 slow clicks already hit the limit	**0 new requests, still signed in**
Reproduction tab after the burst	16 of 16 "Unknown"	**16 partly reproducible, 0 Unknown, 0 Unavailable**
normalized mirrors in the payload	16	**0**
Page payload with manifests loaded (decoded)	390-403 KB	**274 KB (-32%)**
Back, forward, deep link on a fresh load	worked	worked
Refresh data	did not exist	shows "Refreshing...", finishes in about 2 s

Measure	Before (production)	After (production)
Full page load (RSC response / page ready)	about 1.9 s (median click re-ran it)	**2015 ms / 2439 ms, not faster**
Source text in the payload (domain_randomize)	30 copies (60 with manifests)	**30 copies, not removed**



Production after the fix, straight after the rapid-click burst: the Reproduction tab shows 16 partly reproducible generations and no "Unknown". Compare the "before" screenshot in section 2.



Production after the fix: the Lineage tab (URL ?tab=lineage) was on screen 8 ms after the click. Before, the same click showed the previous tab for about two seconds.

> **Honest reading**
>
> Tab clicks: fixed, better than the 100 ms target. The Reproduction failure: gone in this run, but nothing was throttled, so the new failure UI was not exercised on production.
>
> Page load: unchanged at about 2 s. Calls fell from 33 to 18, but the hops did not, so the wait did not shrink. That is what the second change (sections 6 and 7) addresses.

6. The second change: one workspace request

The full page load needed three dependent rounds of backend calls: the session check, then the lists, then the per-run and per-generation data. Fewer calls in the same three rounds did not help, so the change removes a round instead. Three additive backend routes, each built from the services behind the routes it replaces: health and drift for every run of an experiment in one call, and GET /experiments/:id/workspace, which returns the experiment, correlation report, runs (with metrics, evaluations, health and drift) and generations with their manifests in one response. The frontend uses it and falls back to the separate routes on an older backend. The workspace route also sends a Server-Timing header with the duration of each part, and the page logs it.

Measured on a real backend and database

Real backend, real Postgres, a production Next build, seeded like the Phase 1 experiment, with a proxy adding 150 ms of round trip between the two. "Today's production" hides the new routes, which is what the previously deployed backend does. The local database answers in milliseconds, so this shows round-trip effects, not production milliseconds.

Loading mode	Page load	Backend calls	Sequential hops
Today's production (before this change)	648-709 ms	18	3
Batched health and drift only	634-649 ms	14	3

Loading mode	Page load	Backend calls	Sequential hops
One workspace request	**372-402 ms**	**3**	**2**

The three backend calls are the session check, the workspace request, and the page-title lookup, which runs alongside. The page returned by all three modes is byte-for-byte identical, so only the way the data is loaded changed. Wrong-project, missing-experiment and invalid-key requests behave as before. The new route returns exactly what the separate routes return, piece by piece, in an end-to-end test against a real database (243 end-to-end tests pass).

7. After the second deploy: production

Same account and experiment as before.

Measure	Before this change	Now	Change
Server render of the page (5 fetches)	2015 ms	**834-1073 ms, median 871**	**about 2.3x faster**
Hard page load, ready (3 loads)	2439 ms	**1117-1994 ms, median 1604**	**about 1.5x faster**
Refresh data	2074 ms	**1257 ms**	**-40%**
Tab click, UI changed (8 clicks)	8-24 ms	11-24 ms, median 17	unchanged
Data requests per tab click	0	0	unchanged
30-click burst	0 requests, 16 of 16	0 requests, 16 of 16, 0 Unknown	unchanged
Page payload (decoded)	274 KB	274 KB	unchanged

The hard page load varies more (1.1-2.0 s) because it also loads and starts the page's JavaScript, so the server-render figure is the cleaner measure. A few first visits to a tab showed one or two new requests; they were lazy-loaded script files, not data, and revisits showed none.

> **Honest reading**
>
> The full page load is clearly faster, which was the goal. It is not as fast as one hop would be: the session check still runs first, and the workspace request waits for it. Running them together would remove that round (about 230 ms against 380 ms on the local setup). I have not done it because it changes the sign-in flow.
>
> I cannot see the Server-Timing header or the load log line from the browser, so which loading path production used is inferred from the speed-up, not observed. Raw numbers: docs/perf-evidence/after-prod-2/after-production-2.json.

8. Limits and what is not done

- Section 4 numbers use a mock backend; they prove the mechanism, the request counts and the failure handling, not production milliseconds. Section 5 is the production measurement.
- **PERF-02 is largely done.** The second change made it so: the page makes one workspace request instead of 18 calls, in two hops instead of three, and the server render fell from 2.0 s to 0.87 s. What remains is the session check ahead of it. Inside the backend, the workspace route still runs a few database queries per run, so its work grows with the number of runs, but in one request rather than one request per call.
- **PERF-05 is partial.** The payload fell 17% on the mock and 32% on production (403 to 274 KB). The Python source in generation parameters still appears twice per generation (generation and manifest): 30 copies. Removing the manifest copy needs the raw-manifest view to rebuild it, so I left it.
- **Data freshness changed.** Tab switches no longer refetch, so what the SDK records while the page is open appears after "Refresh data" (running training runs still poll live).

- Backend tests: the whole end-to-end suite (243 tests, 18 files) passes against a real Postgres started for the purpose, including a new test that compares the workspace route with the separate routes. The first change's batch route and rate-limit changes are covered by the same run.
- The per-key limit stays at 120/min as before; a render is now about 3 calls, so roughly 40 renders a minute per user (it was about six).
- Deploy order was backend first or together; the frontend works against either. Production now runs both.
- Not verified on production: the Retry banner under real throttling, and the per-user limit with two users. The Server-Timing header and the load log line exist but I cannot read them from the browser, so they have not been looked at yet.

9. Success criteria and result

Criteria set before the deploy, with the result of repeating section 2 on production:

Check	Pass if	Result
Tab click, UI changed (8-click sequence)	under 100 ms each, 0 RSC requests	**Pass: 8-24 ms, 0 requests**
Reproduction tab after rapid clicking	all 16 manifests loaded, no "Unknown" from loading	**Pass: 16 of 16, 0 Unknown**
Initial workspace load	no slower than today; one experiment_workspace.loaded log line	**Pass: faster (server render 871 ms against 2015 ms); the log line itself has not been read from the host**
Refresh data / Retry	reload works; one render request each	**Refresh data: pass. Retry: local build only**

Next: run the session check and the workspace request together, which would remove the last extra round; read the Server-Timing log line to see where the remaining time goes; and optionally remove the duplicate source text from the manifests. None is started.